

DESIGN AND ANALYSIS OF ALGORITHMS

DAY 3 – DYNAMIC PROGRAMMING & PROBLEM SOLVING

Name: N. PARNITA

Register No.: 192525322

Course code: CSA0617

Course Name: Design and Analysis of Algorithm

TABLE OF CONTENTS

Experiment 1: Out of Boundary Paths

Experiment 2: House Robber II

Experiment 3: Climbing Stairs

Experiment 4: Unique Paths in a Grid

Experiment 5: Large Groups in a String

Experiment 6: Game of Life

Experiment 7: Champagne Tower

Experiment 1: Out of Boundary Paths

Objective: To determine the number of ways a ball can move out of an $m \times n$ grid in exactly N steps.

Problem Statement: Given a grid and a starting cell, count the number of paths that move the ball outside the grid boundary in exactly N steps.

Sample Input: $m = 2, n = 2, N = 2, i = 0, j = 0$

Sample Output: 6

Time Complexity: $O(N \times m \times n)$ using dynamic programming.

Explanation: At each step, the number of ways to reach every cell is stored. From each cell, moves in four directions are considered. A move outside the grid contributes to the answer.

Python Program

```
def find_paths(m, n, N, start_row, start_col):
    dp = [[0] * n for _ in range(m)]
    dp[start_row][start_col] = 1
    result = 0

    directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]

    for _ in range(N):
        next_dp = [[0] * n for _ in range(m)]

        for r in range(m):
            for c in range(n):
                if dp[r][c] == 0:
                    continue

                for dr, dc in directions:
                    nr, nc = r + dr, c + dc

                    if nr < 0 or nr >= m or nc < 0 or nc >= n:
                        result += dp[r][c]
                    else:
                        next_dp[nr][nc] += dp[r][c]

        dp = next_dp

    return result

m = int(input("Enter m: "))
n = int(input("Enter n: "))
N = int(input("Enter N: "))
i = int(input("Enter starting row i: "))
j = int(input("Enter starting column j: "))

print("Number of ways =", find_paths(m, n, N, i, j))
```

Execution / Output

```
Enter m: 2
Enter n: 2
Enter N: 2
Enter starting row i: 0
Enter starting column j: 0
Number of ways = 6
```

Result: The program was implemented and the expected result was obtained.

Experiment 2: House Robber II

Objective: To find the maximum amount of money that can be robbed from houses arranged in a circle without robbing adjacent houses.

Problem Statement: Given a circular array of house values, find the maximum money that can be robbed without triggering the security system.

Sample Input: nums = [2, 3, 2]

Sample Output: 3

Time Complexity: $O(n)$ time and $O(1)$ auxiliary space.

Explanation: Because the houses form a circle, the first and last houses cannot both be selected. The problem is divided into two linear cases: exclude the first house or exclude the last house.

Python Program

```
def rob_linear(nums):
    prev = 0
    curr = 0

    for money in nums:
        prev, curr = curr, max(curr, prev + money)

    return curr

def rob_circle(nums):
    if len(nums) == 1:
        return nums[0]

    return max(rob_linear(nums[:-1]), rob_linear(nums[1:]))

nums = list(map(int, input("Enter house amounts: ").split()))

print("Maximum money =", rob_circle(nums))
```

Execution / Output

```
Enter house amounts: 2 3 2
Maximum money = 3
```

Result: The program was implemented and the expected result was obtained.

Experiment 3: Climbing Stairs

Objective: To find the number of distinct ways to climb n stairs when one can climb either 1 or 2 steps at a time.

Problem Statement: Given n steps, calculate the number of distinct ways to reach the top.

Sample Input: $n = 4$

Sample Output: 5

Time Complexity: $O(n)$ time and $O(1)$ auxiliary space.

Explanation: The number of ways to reach step n is the sum of the ways to reach steps $n-1$ and $n-2$. This gives the same recurrence as the Fibonacci sequence.

Python Program

```
def climb_stairs(n):
    if n <= 2:
        return n

    a, b = 1, 2

    for _ in range(3, n + 1):
        a, b = b, a + b

    return b

n = int(input("Enter n: "))
print("Number of ways =", climb_stairs(n))
```

Execution / Output

```
Enter n: 4
Number of ways = 5
```

Result: The program was implemented and the expected result was obtained.

Experiment 4: Unique Paths in a Grid

Objective: To calculate the number of unique paths from the top-left corner to the bottom-right corner of a grid.

Problem Statement: A robot can move only right or down in an $m \times n$ grid. Find the number of unique paths to the destination.

Sample Input: $m = 7, n = 3$

Sample Output: 28

Time Complexity: $O(m \times n)$ time and $O(n)$ auxiliary space using one-dimensional dynamic programming.

Explanation: Each cell stores the number of ways to reach it. The number of paths to a cell is obtained from the cell above and the cell to its left.

Python Program

```
def unique_paths(m, n):
    dp = [1] * n

    for _ in range(1, m):
        for j in range(1, n):
            dp[j] += dp[j - 1]

    return dp[-1]

m = int(input("Enter m: "))
n = int(input("Enter n: "))

print("Number of unique paths =", unique_paths(m, n))
```

Execution / Output

```
Enter m: 7
Enter n: 3
Number of unique paths = 28
```

Result: The program was implemented and the expected result was obtained.

Experiment 5: Large Groups in a String

Objective: To find all groups of three or more consecutive identical characters in a string.

Problem Statement: Given a string of lowercase letters, return the intervals [start, end] of every large group, sorted by start index.

Sample Input: s = "abbxxxxzzy"

Sample Output: [[3, 6]]

Time Complexity: O(n) time and O(k) output space, where k is the number of large groups.

Explanation: The string is scanned from left to right. Whenever a consecutive group ends, its length is checked. If the group contains at least three characters, its starting and ending indices are recorded.

Python Program

```
def large_groups(s):
    result = []
    start = 0

    for i in range(1, len(s) + 1):
        if i == len(s) or s[i] != s[start]:
            if i - start >= 3:
                result.append([start, i - 1])
            start = i

    return result

s = input("Enter string: ")
print("Large groups =", large_groups(s))
```

Execution / Output

```
Enter string: abbxxxxzzy
Large groups = [[3, 6]]
```

Result: The program was implemented and the expected result was obtained.

Experiment 6: Game of Life

Objective: To compute the next state of a Game of Life board using the given cellular automaton rules.

Problem Statement: Given the current $m \times n$ board, apply the four rules simultaneously to every cell and return the next state.

Sample Input: board = [[0,1,0],[0,0,1],[1,1,1],[0,0,0]]

Sample Output: [[0,0,0],[1,0,1],[0,1,1],[0,1,0]]

Time Complexity: $O(m \times n)$ time and $O(m \times n)$ auxiliary space in this straightforward implementation.

Explanation: Each cell is examined along with its eight neighboring cells. The next state is stored separately so that all changes are based on the original board state.

Python Program

```
def game_of_life(board):
    rows = len(board)
    cols = len(board[0])
    next_board = [[0] * cols for _ in range(rows)]

    for r in range(rows):
        for c in range(cols):
            live_neighbors = 0

            for dr in (-1, 0, 1):
                for dc in (-1, 0, 1):
                    if dr == 0 and dc == 0:
                        continue

                    nr, nc = r + dr, c + dc

                    if 0 <= nr < rows and 0 <= nc < cols:
                        live_neighbors += board[nr][nc]

            if board[r][c] == 1:
                if live_neighbors == 2 or live_neighbors == 3:
                    next_board[r][c] = 1
            else:
                if live_neighbors == 3:
                    next_board[r][c] = 1

    return next_board

board = [
    [0, 1, 0],
    [0, 0, 1],
    [1, 1, 1],
    [0, 0, 0]
]

print(game_of_life(board))
```

Execution / Output

```
[[0, 0, 0], [1, 0, 1], [0, 1, 1], [0, 1, 0]]
```

Result: The program was implemented and the expected result was obtained.

Experiment 7: Champagne Tower

Objective: To determine how full a particular glass is after pouring champagne into a pyramid-shaped tower.

Problem Statement: Given the number of cups poured and the queried row and glass, return how full that glass is. Rows and glasses are 0-indexed.

Sample Input: poured = 2, query_row = 1, query_glass = 1

Sample Output: 0.50000

Time Complexity: $O(\text{query_row}^2)$ time and $O(\text{query_row})$ auxiliary space.

Explanation: Champagne is distributed from each glass equally to the two glasses below it whenever the amount exceeds one cup. Only the rows up to the queried row need to be simulated.

Python Program

```
def champagne_tower(poured, query_row, query_glass):
    row = [float(poured)]

    for _ in range(query_row):
        next_row = [0.0] * (len(row) + 1)

        for j, amount in enumerate(row):
            excess = max(0.0, amount - 1.0)
            next_row[j] += excess / 2
            next_row[j + 1] += excess / 2

        row = next_row

    return min(1.0, row[query_glass])

poured = int(input("Enter poured cups: "))
query_row = int(input("Enter query row: "))
query_glass = int(input("Enter query glass: "))

print(f"{champagne_tower(poured, query_row, query_glass):.5f}")
```

Execution / Output

```
Enter poured cups: 2
Enter query row: 1
Enter query glass: 1
0.50000
```

Result: The program was implemented and the expected result was obtained.